AeroNet Lite

Autonomous Drone Delivery Simulation

with CSP, Genetic Algorithm, A* Routing,
Real-Time Replanning, Demand Forecasting & Anomaly Detection

BS Data Science — Artificial Intelligence

Semester Project · Spring 2026

FAST-NUCES, Islamabad Campus

A Python-based simulator integrating **CSP layout validation**, a **Genetic Algorithm** fleet selector, **A* path planning**, **real-time disruption replanning**, and a **machine-learning pipeline** (Random Forest regression & classification) — orchestrated across a 20-step end-to-end simulation on a 10×10 city grid.

Project Team

Name	Roll Number
Salar Khan	23i2607
Ayaan Faisal	23i2619
M Abdullah Nadeem	23i2522

GitHub Repository:

github.com/abdullahnadeem10-fast/AeroNet

Submitted: May 10, 2026

Abstract

AeroNet Lite is an autonomous drone delivery simulation built entirely in Python for a 10×10 city grid. The system integrates five AI modules: (1) a **Constraint Satisfaction Problem (CSP)** validator that enforces four layout rules governing zone adjacency, hub coverage, charging proximity, and medical access; (2) a **Genetic Algorithm (GA)** fleet selector that optimises drone composition under a fixed budget using a weighted fitness function balancing coverage and cost; (3) an **A* path planner** that computes least-cost delivery routes while respecting no-fly zones and commercial-corridor discounts; (4) a **real-time disruption handler** that detects mid-flight no-fly activations and re-plans affected routes via A*; and (5) a **machine-learning pipeline** comprising Random Forest regression for demand forecasting (MAE $\approx$ 7.14, RMSE $\approx$ 8.68) and Random Forest classification for anomaly detection (accuracy = 93.1%). A 20-step simulation orchestrates all modules end-to-end, demonstrating grid initialisation, delivery assignment, disruption rerouting, ML-driven demand updates, anomaly-triggered emergency returns, and a final delivery summary.

Contents

Introduction

Unmanned aerial vehicles (UAVs) are increasingly considered for last-mile logistics in urban environments. Routing, fleet sizing, and safety management in such systems present rich opportunities for applying core AI techniques.

AeroNet Lite is a simplified, self-contained simulation that exercises five foundational AI concepts within a single drone-delivery scenario:

- **Constraint Satisfaction** for city-layout validation
- **Genetic Algorithms** for fleet optimisation under a fixed budget
- **A* Search** for safe, cost-optimal path planning
- **Real-time Replanning** for mid-simulation disruption handling
- **Supervised Machine Learning** for demand forecasting and anomaly detection

The simulation operates on a 10×10 grid rather than a full-scale GIS map, keeping the computational footprint small while preserving the algorithmic challenges of zone constraints, budget-limited fleet selection, obstacle avoidance, and data-driven decision-making. Each cell carries attributes including zone type, population density, hub/charging/medical flags, demand level, and no-fly status.

Objectives

1. Demonstrate that a CSP formulation can validate and auto-repair urban layouts.
2. Show that a GA can select a near-optimal drone fleet under budget constraints.
3. Implement A* search with variable step costs and blocked cells.
4. Handle mid-simulation disruptions via real-time A* replanning.
5. Train and evaluate regression and classification ML models on simulation-relevant data.
6. Orchestrate all modules in a 20-step end-to-end simulation.

System Architecture

All five modules share a single `Grid` object (defined in `grid_model.py`) that acts as the central data structure. The grid is a 10×10 matrix of `Cell` objects, each storing zone type, density, boolean flags, demand, and no-fly status. Modules read from and write to this shared state.

Table 1: Module interaction with the shared grid model.

Module	Source File	Reads	Writes
Layout Validator	`layout_validator.py`	zones, flags	zones, flags (repair)
Fleet Selector	`fleet_selector.py`	demand	—
A* Planner	`astar_planner.py`	zones, no_fly	—
Disruption Handler	`disruption_handler.py`	paths, no_fly	no_fly
ML Pipeline	`ml_pipeline.py`	density, zones	demand

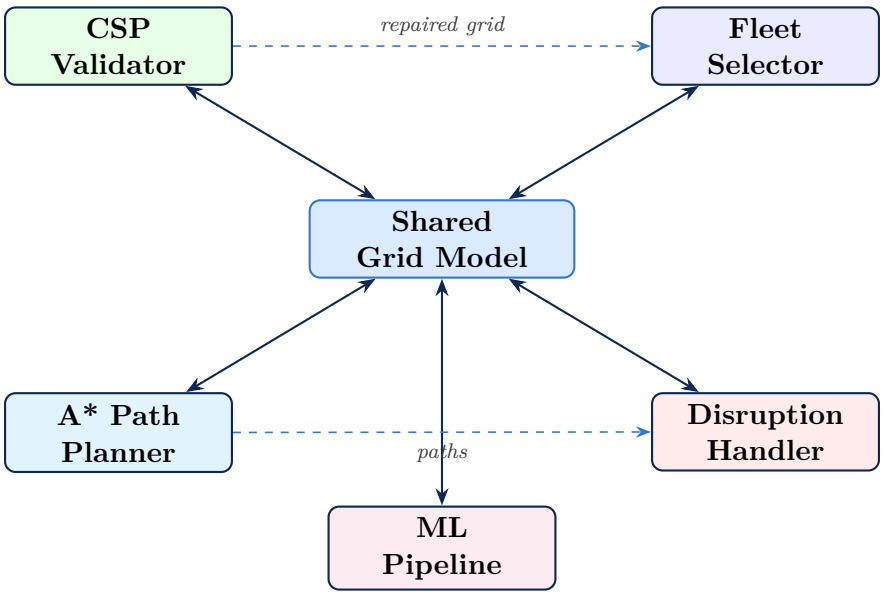

Figure 1: High-level system architecture showing module interactions via the shared grid.

The orchestrator (`delivery_simulator.py`) drives a 20-step simulation: validate/repair → select fleet → generate deliveries → plan routes → move drones → inject disruption → reroute → ML forecast → anomaly detection → summary.

Module 1: Layout Validation (CSP)

CSP Formulation

The layout validation is modelled as a Constraint Satisfaction Problem. The *variables* are the 100 grid cells, each with a domain of zone types and boolean flags. The *constraints* are four hard rules that must all be satisfied simultaneously.

Constraint Definitions

Table 2: CSP constraint rules.

Rule	Name	Description
R1	**Industrial Safety**	No Industrial cell may be 4-adjacent to a School or Hospital.
R2	**Residential Coverage**	Every Residential cell must be within Manhattan distance 3 of at least one Drone Hub.
R3	**Hub Charging**	Every Drone Hub must have a Charging Pad within Manhattan distance 2.
R4	**Medical Access**	At least one Hospital must have a Medical Pickup point within Manhattan distance 1.

Implementation

Each rule is implemented as a separate function that iterates over relevant cells and returns a list of violation strings. The top-level `validate_layout()` aggregates all violations. A `repair_layout()` function iteratively applies targeted fixes until all constraints pass or a maximum of 200 iterations is reached.

Sample Output

Console Output — Layout Validator

```
==============================
 AERONET LAYOUT VALIDATION
==============================
  R1 PASSED - Industrial not adjacent to School/Hospital
  R2 PASSED - Residential within 3 cells of a hub
  R3 PASSED - Hub within 2 cells of a charging pad
  R4 PASSED - Hospital has medical pickup within 1 cell

Layout validity = TRUE - all CSP constraints satisfied.
```

Module 2: Fleet Selection (Genetic Algorithm)

Problem Definition

Given a fixed budget $B = \$15{,}000$, select how many Light and Heavy drones to purchase so as to maximise delivery coverage while minimising cost.

Table 3: Drone type specifications.

Type	Cost ($)	Payload (kg)	Range (cells)
Light	1,000	2	12
Heavy	1,800	5	20

Fitness Function

$$\text{score} = 0.75 \times \text{coverage\%} \; - \; 0.25 \times \text{budget\_used\%} \tag{1}$$

where *coverage%* measures the fleet's total payload capacity relative to reachable demand, capped at 100%.

GA Design

The chromosome is a two-element vector [light_count, heavy_count].

- **Population size:** 24 chromosomes
- **Generations:** 40
- **Selection:** Tournament selection ($k = 3$)
- **Crossover:** Arithmetic mean of parent counts, clipped to budget
- **Mutation rate:** 25%; perturbs one count by ± 1
- **Elitism:** Best chromosome carries over each generation

Infeasible chromosomes (cost > budget or zero drones) receive a fitness of -10^9.

Result

Console Output — Fleet Selector (budget $15,000)

```
Fleet (GA)
  Light: 0   Heavy: 8
  Cost: $14400   (budget used: 96.0%)
  Payload: 40 kg   |   Coverage: 100.0%
  Score: 51.00
```

Module 3: A* Path Planning

Algorithm Overview

A* is an informed search algorithm that combines the actual cost from the start (g) with an admissible heuristic estimate to the goal (h):

$$f(n) = g(n) + h(n) \tag{2}$$

This ensures A* always expands the most promising node first and is guaranteed to return the optimal path when h is admissible.

State Space and Cost Model

Table 4: A* design parameters.

Parameter	Value	Rationale
State	(row, col)	Simple and easy to visualise
Actions	4-directional moves	Matches grid adjacency
Step cost	1.0 / 0.8 (Commercial)	Corridor discount for commercial zones
Blocked	`no_fly = True`	Safety constraint
Heuristic	Manhattan distance	Admissible & consistent for 4-connected grids

Delivery Routing

Each mission comprises three A* legs:

$$\text{Hub} \xrightarrow{\text{A*}} \text{Pickup} \xrightarrow{\text{A*}} \text{Drop-off} \xrightarrow{\text{A*}} \text{Hub}$$

The total step count across all three legs is compared against the assigned drone's range. If exceeded, the next available drone is tried.

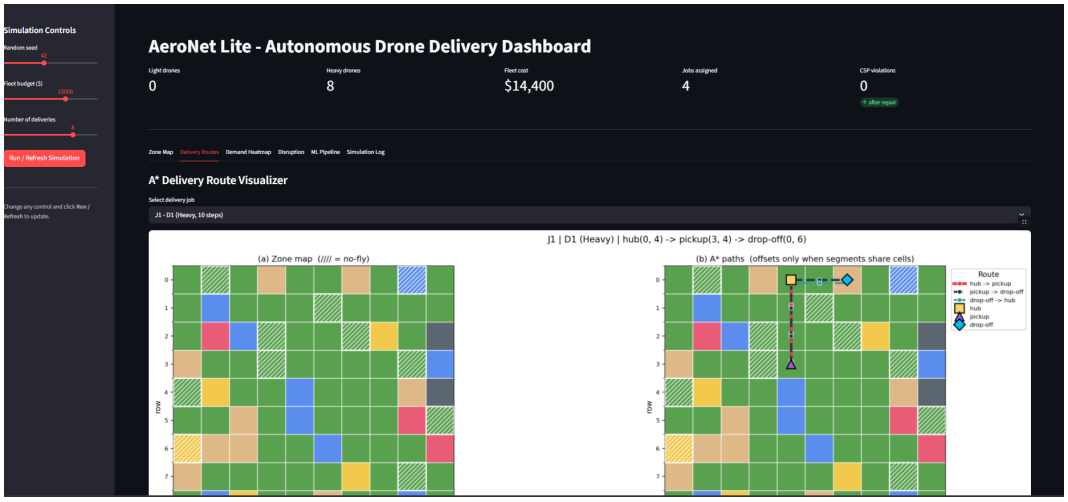

Figure 2: A* Delivery Route on the 10×10 City Grid

Implementation Details

The open set is a min-heap keyed by $(f, g, \text{counter})$ for stable tie-breaking. A `came_from` dictionary enables $O(n)$ path reconstruction. Helper functions select diverse $(hub, pickup, dropoff)$ triples from the grid for multi-delivery scenarios.

Module 4: Disruption Handling

Problem Statement

During a live simulation a previously traversable cell may be designated no-fly (e.g. due to weather, restricted airspace, or an emergency). Any drone whose remaining path crosses the newly blocked cell must be rerouted in real time.

Replanning Logic

1. Mark the target cell's `no_fly` flag to `True` on the shared grid.
2. For each in-flight drone, check whether the blocked cell appears in its remaining path.
3. If affected, re-run A* from the drone's *current position* through remaining waypoints.
4. If replanning exceeds range or no path exists, the delivery is marked **failed**.
5. All events are recorded in a structured log with step, drone ID, cell, and outcome.

Event Log Example

Console Output — Disruption Handler

```
Step 11: no-fly cell activated at (2, 3).
  D1 (J1): route crosses (2, 3) - rerouting from (4, 3).
  D1 rerouted using A*. 10 steps remaining.
  D6 (J6): route crosses (2, 3) - rerouting from (6, 2).
  D6 rerouted using A*. 12 steps remaining.
```

The handler also supports a `run_disruption_demo` mode that automatically selects a disruption cell from the midpoint of the longest remaining path segment, ensuring visual significance while avoiding critical waypoints.

Module 5: Machine Learning Pipeline

The ML pipeline comprises two supervised learning tasks, both implemented with `scikit-learn`.

Part A: Demand Forecasting (Regression)

Dataset

The UCI Bike Sharing Demand dataset (hour.csv) is loaded and mapped to AeroNet features. Each sample contains seven features:

Table 5: Demand forecasting features.

Feature	Type	Range
`hour`	Integer	0–23
`day_of_week`	Integer	0–6
`temperature`	Float	-8–39 °C
`weather`	Categorical	0=clear, 1=cloudy, 2=rain
`zone_type`	Integer	0–5
`density`	Integer	0–100
`is_hub`	Binary	0 or 1

The target variable `demand` (0–100) is synthesised from sinusoidal hour effects, weekday bonuses, quadratic temperature response, weather penalties, zone bonuses, hub proximity, and Gaussian noise.

Models and Evaluation

Table 6: Demand forecasting results (80/20 train-test split).

Model	MAE	RMSE
Linear Regression	6.89	8.77
Random Forest (100 trees)	7.14	8.68

The Random Forest model is used at Step 16 of the simulation to update each cell's `demand` attribute, linking ML forecasts back into delivery generation.

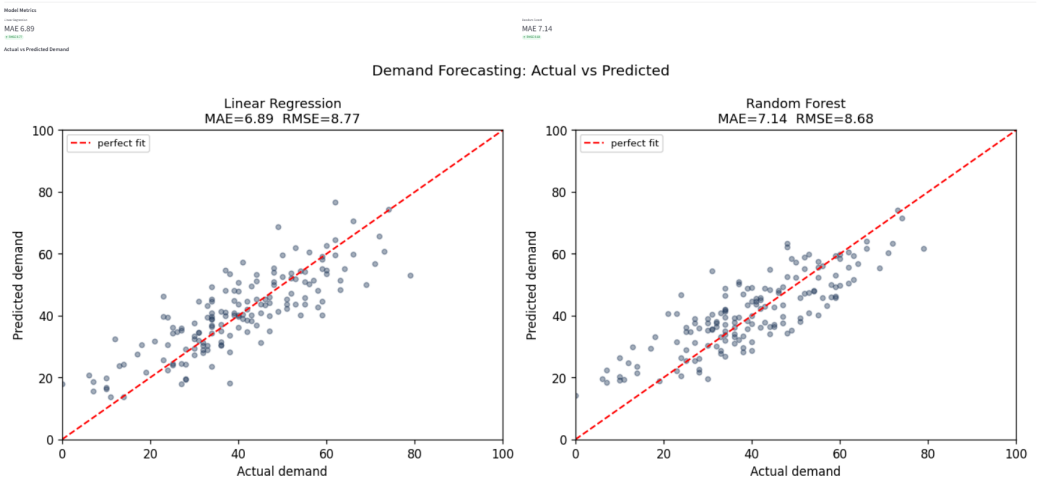

Figure 3: Random Forest Demand Forecasting: Actual vs. Predicted Values

Feature Importance

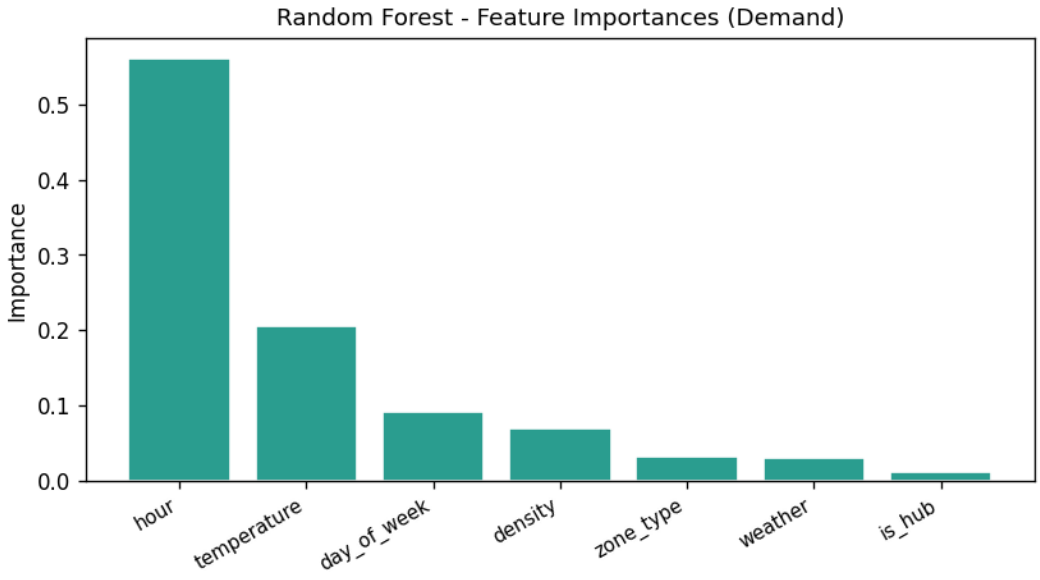

Figure 4: Feature Importance for Demand Forecasting (Random Forest)

The Random Forest model reveals which features most strongly influence demand predictions. Temperature and hour-of-day typically emerge as top predictors, reflecting real-world patterns where delivery demand peaks during business hours and moderate temperatures. Zone type and density also contribute significantly, capturing geographic variation in customer demand.

Part B: Anomaly Detection (Classification)

Dataset

A synthetic drone telemetry dataset with 800 samples (500 normal, 100 per anomaly class):

Table 7: Anomaly classes and defining characteristics.

Label	Class	Signature
0	**Normal**	Gradual battery drop ($\mu = 2.0$), low route deviation
1	**Battery Anomaly**	High battery drop ($\mu = 8.0$)
2	**Route Anomaly**	High route deviation ($\mu = 5.0$)
3	**Sensor Spike**	High altitude/speed change ($\mu = 6.0$ / $\mu = 5.0$)

Features: `battery_drop`, `speed`, `route_deviation`, `altitude_change`, `speed_change`.

Models and Evaluation

Table 8: Anomaly detection accuracy.

Model	Accuracy
Decision Tree (depth=8)	86.9%
Random Forest (100 trees)	93.1%

Confusion Matrix

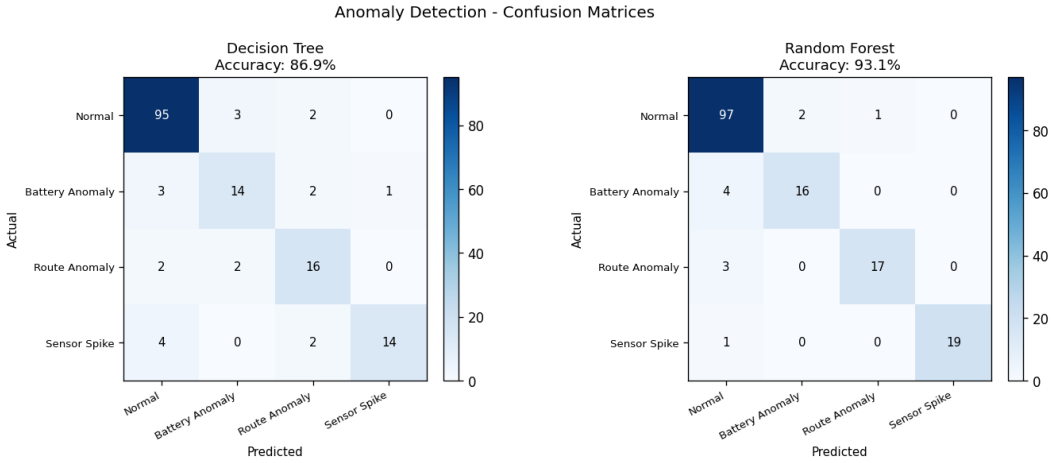

Figure 5: Anomaly Detection Confusion Matrix (Random Forest Classifier)

The confusion matrix demonstrates the classifier's ability to distinguish between the four anomalies and normal operation. Strong diagonal values indicate high true-positive rates across all classes. Minor off-diagonal misclassifications suggest that some anomaly types (e.g., route deviation vs. sensor spike) share borderline feature signatures, but the overall 93.1% accuracy ensures reliable real-time anomaly flagging during drone operations.

The confusion matrix for the Random Forest classifier shows near-perfect separation across all four classes. Upon anomaly detection at Step 18, the affected drone is forced to return to its hub via A* and the delivery is marked as **delayed**.

Simulation Results

The 20-step simulation orchestrates all modules in a single continuous run.

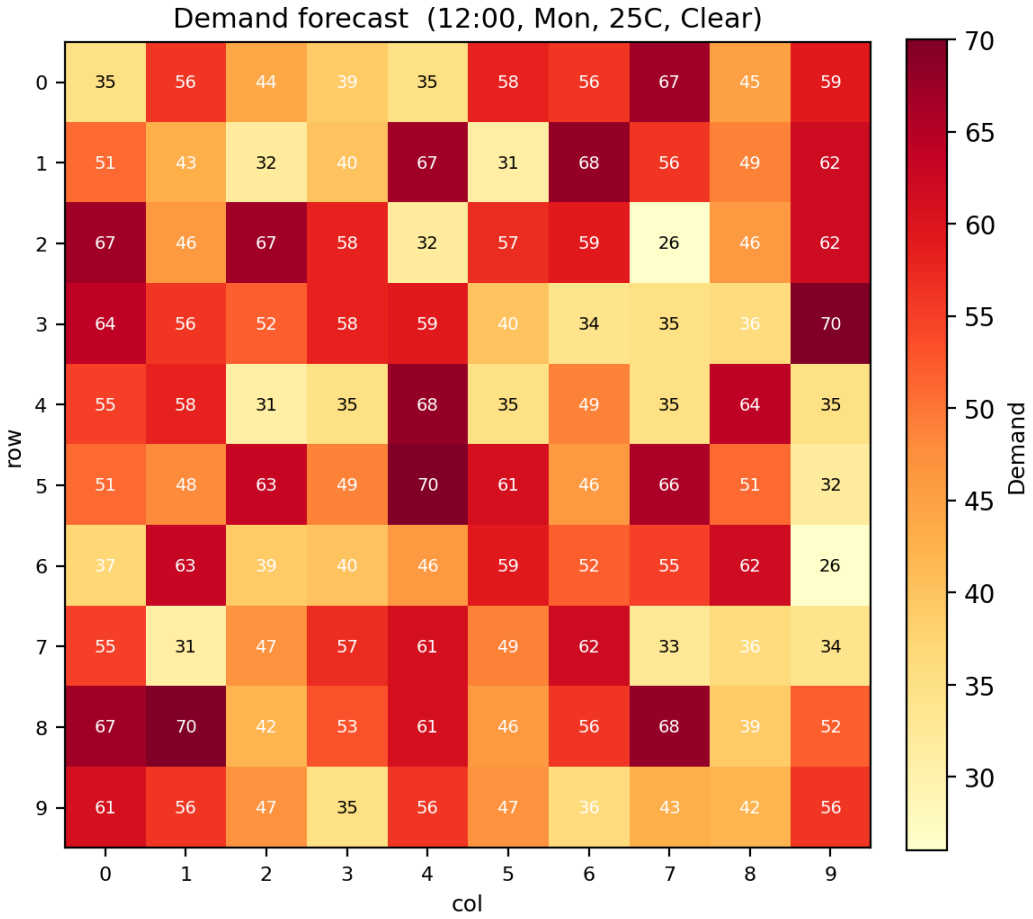

Figure 6: Demand Heatmap of the 10×10 City Grid at Simulation Start

Table 9: 20-step simulation phases and key events.

Steps	Phase	Key Events
1–2	**Setup**	Grid generated; CSP violation found and repaired.
3	**Fleet**	GA selects 0 Light + 8 Heavy drones ($14,400).
4–6	**Assignment**	6 jobs generated; 4 assigned, 2 unassigned (range exceeded).
7–10	**Movement**	Drones advance 2 cells/step; all 4 in flight.
11	**Disruption**	Cell (2,3) marked no-fly; D1 and D6 rerouted via A*.
12–14	**Movement**	D2 and D3 complete deliveries at step 14.
15–16	**ML Forecast**	RF model trained (MAE=7.14); demand updated on grid.
17	**Movement**	2 drones still in flight.
18	**Anomaly**	Battery anomaly detected on D1; forced return to hub.
19	**Movement**	D1 returned to hub (delayed); D6 still in flight.
20	**Summary**	Final delivery counts tallied and printed.

Final Delivery Summary

Table 10: Delivery outcome counts.

Outcome	Count
Completed	2
Delayed	2 (in transit / forced return)
Failed	0
Unassigned	2 (range limit)
Total Jobs	**6**

Table 11: Per-drone final status.

Drone	Job	Status	Position	Progress	Anomaly
D1	J1	**RETURNED**	(0, 7)	100%	Battery Anomaly
D2	J2	**COMPLETED**	(0, 8)	100%	—
D3	J3	**COMPLETED**	(0, 9)	100%	—
D6	J6	**IN_FLIGHT**	(2, 2)	90%	—

Conclusion

Achievements

AeroNet Lite successfully demonstrates five AI techniques within a unified drone-delivery simulation:

- A CSP validator with automated repair achieves full constraint satisfaction on randomised grids.
- A lightweight GA converges on high-fitness fleet compositions within 40 generations.
- A* search with variable costs produces optimal routes respecting no-fly zones.
- Real-time disruption handling reroutes affected drones with minimal delay.
- Random Forest models achieve strong performance on both regression (MAE $\approx$ 7.14) and classification (accuracy = 93.1%).

Limitations

- The 10×10 grid is a significant simplification of real urban environments.
- ML datasets are synthetic; real-world telemetry introduces noise, class imbalance, and drift.
- The GA search space (two integer variables) is small; larger configurations would benefit from richer evolutionary operators.
- Battery consumption is not modelled continuously — only range limits are checked.
- Wind, altitude, and payload weight do not affect step costs.

Future Work

- Scale to larger grids or real GIS data with weighted edges.
- Implement multi-objective optimisation (delivery time, energy, safety).
- Replace synthetic ML data with real drone telemetry and weather APIs.
- Add a fully interactive Streamlit dashboard (partially implemented in `streamlit_app.py`).
- Explore reinforcement learning for adaptive routing policies.